

Java – Domande e Risposte per il Colloquio Tecnico

[Basato su: *Java – Supporto al Corso* (Ferdinando Primerano, Stefano Rubinetti, 2020/2023)]

1. Basi della Programmazione

D: Cos'è un programma? Danne almeno tre definizioni.

R:

- **Lista di istruzioni:** comandi dati alla macchina in sequenza.
- **Traduzione di un algoritmo:** un algoritmo è una serie ordinata di passi per arrivare a un risultato; il programma lo traduce in un linguaggio comprensibile alla macchina.
- **Funzione matematica:** $O = P(I)$, dove P trasforma degli input in output.
- **Motore di pezzi interconnessi:** componenti che lavorano insieme per produrre un risultato (visione orientata agli oggetti).

D: Qual è la differenza tra file `.java` e file `.class`?

R: Il file `.java` è il codice sorgente scritto dal programmatore. Il file `.class` è il codice compilato (pseudo-compilato in bytecode) che viene eseguito dalla Java Virtual Machine (JVM). Eclipse esegue automaticamente questo processo di compilazione al "play".

D: Cos'è uno scope (blocco di codice)?

R: Un blocco di codice è un'area delimitata da `{ }`. Ogni blocco definisce uno scope: le variabili dichiarate in uno scope sono visibili in esso e in tutti i blocchi interni (scope annidati), ma non all'esterno. Non possono esistere due variabili con lo stesso nome nello stesso scope.

D: Quante e quali sono le tre forme principali di istruzione in Java?

R:

1. **Dichiarazione di variabile:** `int v;`
2. **Assegnamento:** `v = 5;`
3. **Chiamata a metodo:** `System.out.println(v);`

D: Cos'è una variabile e quali regole fondamentali la governano?

R: Una variabile è un'area di memoria con un nome e un tipo. Regole:

- Il tipo è immutabile (tipizzazione statica): `int v` rimarrà sempre `int`.
- Non si possono avere due variabili con lo stesso nome nello stesso scope.
- Va inizializzata prima di essere usata.

D: Cos'è la concordanza dei tipi (Type Mismatch)?

R: In un assegnamento `a = b`, `b` deve essere dello stesso tipo di `a` o di un tipo compatibile. Esempi errati: `int a = 7.1;` oppure `String s = 7;`. Eccezione: un `int` può essere assegnato a un `double` (è un caso particolare), ma non il contrario senza cast esplicito.

D: Cos'è un'espressione e che tipo ha?

R: Un'espressione è tutto ciò che restituisce un valore. Ha sempre un tipo. Esempi: `base * altezza` (`int`), `eta > 18` (`boolean`), `"Casa in " + via` (`String`). Il tipo dell'espressione non è necessariamente il tipo delle sue componenti (es: `eta > 18` usa due `int` ma produce un `boolean`).

D: Cosa sono le condizioni composte? Spiega `&&`, `||` e `!`.

R:

- `&&` (AND): vera se **entrambe** le condizioni sono vere.
- `||` (OR): vera se **almeno una** delle condizioni è vera.
- `!` (NOT): inverte il valore booleano (`!true → false`).

Esempio: `genere.equals("M") && statura > 175` è vera solo se la persona è maschio E alta più di 175 cm.

D: Qual è la differenza tra `do-while` e `while`?

R:

- `do-while`: il corpo è eseguito **almeno una volta**; la condizione è verificata **alla fine**.
- `while`: la condizione è verificata **prima** della prima esecuzione; il ciclo potrebbe non essere mai eseguito.

D: Qual è la struttura del ciclo `for` e cosa contiene?

R:

```
for(inizializzazione; condizione_di_ripetizione; aggiornamento) { // corpo }
```

- **Inizializzazione**: eseguita una sola volta all'inizio.
- **Condizione**: verificata prima di ogni iterazione (ciclo ad esecuzione opzionale).
- **Aggiornamento**: eseguito alla fine di ogni iterazione.

D: A cosa servono `break` e `continue`?

R:

- `continue`: salta l'iterazione corrente e passa alla successiva.
- `break`: interrompe completamente il ciclo, indipendentemente dalla condizione. Utile per terminare una ricerca appena trovato il risultato.

D: Cos'è l'operatore ternario? Quando si usa?

R: `variabile = condizione ? valore_se_vera : valore_se_falsa;`

Si usa per assegnare uno di due valori a una variabile in base a una condizione binaria. È più conciso di un `if-else` quando si tratta solo di scegliere un valore.

D: A cosa serve il costrutto `switch`?

R: Serve a gestire una casistica discreta, cioè più casi diversi relativi ai valori di una singola variabile. È più leggibile di una serie di `if` quando i casi sono molti. Il `break` alla fine di ogni `case` impedisce il "fall-through" (l'esecuzione del caso successivo). Il `default` viene eseguito se nessun caso coincide.

2. Metodi

D: Cos'è un metodo? Qual è la differenza tra metodo `void` e non `void`?

R: Un metodo è un **sottoprogramma** che appartiene a una classe o a un oggetto. Non esiste mai "libero".

- **Void:** esegue un'azione ma non restituisce alcun valore (es: `println`).
- **Non void:** produce (restituisce) un valore. Il tipo di ritorno è dichiarato prima del nome del metodo (es: `int sum(int a, int b)`).

D: Cosa fa la parola chiave `return`?

R: Nei metodi non void, `return` è **obbligatoria** e restituisce un valore al chiamante, terminando l'esecuzione del metodo. Nei metodi void è opzionale e serve solo per uscire anticipatamente dal metodo. Una volta eseguito `return`, nessun'altra istruzione del metodo viene eseguita.

D: Qual è la differenza tra metodi `static` e non `static`?

R:

- **Static (di classe):** appartiene alla classe, non all'oggetto. Si chiama con `NomeClasse.metodo()`. Può accedere solo a variabili e metodi statici.
- **Non static (di oggetto):** appartiene a un'istanza specifica. Si chiama con `oggetto.metodo()`. Può accedere alle proprietà dell'oggetto (`this`).

D: Cos'è l'overloading di un metodo?

R: L'overloading permette di definire più metodi con lo stesso nome ma con **firme diverse** (numero o tipo di parametri diversi). Il compilatore sceglie quale versione chiamare in base agli argomenti passati. Non si possono sovraccaricare metodi che differiscono solo nel tipo di ritorno.

3. Classi e Oggetti

D: Cos'è una classe e cos'è un oggetto? Qual è la differenza?

R:

- **Classe:** è il modello/tipo, lo "stampo" da cui creare oggetti. Definisce le proprietà e i comportamenti comuni. (es: `Person`)
- **Oggetto:** è un'istanza concreta della classe. Ogni oggetto ha il proprio stato. (es: `ferdinando` è un'istanza di `Person`)

D: Cosa fa l'operatore `new`? Cos'è un costruttore?

R: `new` è l'operatore che **istanzia** una classe, allocando la memoria necessaria per l'oggetto. Invoca sempre un **costruttore**: un metodo speciale con lo stesso nome della classe e senza tipo di ritorno. Se non si scrive alcun costruttore, Java fornisce un costruttore di default (implicito) senza parametri. Se si definisce un costruttore esplicito, quello implicito viene disabilitato.

D: Cos'è `null`?

R: `null` indica l'**assenza di un oggetto**. Una variabile riferimento dichiarata ma non inizializzata vale `null`. Non è zero, non è stringa vuota: è proprio "nessun oggetto". Chiamare un metodo su una variabile `null` genera una `NullPointerException`.

D: Cos'è l'incapsulamento? Come si realizza in Java?

R: L'incapsulamento è il principio di **nascondere lo stato interno** di un oggetto e di permetterne l'accesso solo tramite metodi controllati. Si realizza:

- Dichiarando le proprietà `private` (invisibili dall'esterno).
- Fornendo metodi `public` **getter** (per leggere) e **setter** (per modificare) che possono includere validazioni.

Esempio:

```
private int age; public int getAge() { return age; } public void setAge(int a) { if(a >= 0) this.age = a; }
```

D: Cos'è `this`?

R: `this` è un riferimento all'oggetto **corrente** su cui il metodo è stato invocato. È utile per distinguere le proprietà dell'oggetto dai parametri del metodo quando hanno lo stesso nome:

```
public Person(String name) { this.name = name; // this.name è la proprietà, name è il parametro }
```

D: Cosa sono le variabili di classe (`static`)? E i metodi di classe?

R:

- **Variabile di classe (`static`)**: appartiene alla classe, non all'istanza. Esiste una sola copia condivisa da tutti gli oggetti. Es: `static int contatore`;
- **Metodo di classe (`static`)**: non può accedere alle proprietà non statiche dell'oggetto. Può essere chiamato senza istanziare la classe.

D: Cos'è il Garbage Collector?

R: È il meccanismo automatico di Java che **libera la memoria** occupata dagli oggetti che non sono più raggiungibili da nessuna variabile del programma. Il programmatore non deve deallocare manualmente la memoria (come in C/C++). Non è deterministico: non si sa esattamente quando verrà eseguito.

4. Vettori (Array)

D: Cos'è un vettore (array) in Java? Come si dichiara e si crea?

R: Un vettore è una struttura dati che contiene un numero **fisso** di elementi dello stesso tipo, ordinati e accessibili tramite indice (da 0 a `length-1`).

```
int[] v; // dichiarazione v = new int[10]; // creazione (alloca 10 interi)
v[0] = 5; // accesso al primo elemento int len = v.length; // lunghezza del vettore
```

D: Come si scorre un vettore? Quanti modi esistono?

R:

- **For classico** (con indice):

```
for(int i = 0; i < v.length; i++) { System.out.println(v[i]); }
```

- **For-each**:

```
for(int el : v) { System.out.println(el); }
```

D: Cosa sono le operazioni `map`, `filter` e `reduce` su un vettore?

R:

- **Map (trasformazione 1-1)**: trasforma ogni elemento del vettore in un nuovo valore. Il vettore risultante ha la stessa dimensione.
- **Filter (filtraggio)**: seleziona solo gli elementi che soddisfano una condizione. Il vettore risultante è più piccolo o uguale.
- **Reduce (riduzione)**: riduce l'intero vettore a un singolo valore (es: somma, massimo, minimo).

5. Oggetto Aggregatore

D: Cos'è l'oggetto aggregatore? Qual è il suo ruolo nell'architettura?

R: L'oggetto aggregatore è una classe che **contiene** e **gestisce** una collezione di oggetti di altra classe (es: un `Census` che gestisce un array di `Person`). Usa la metafora del motore: ogni parte fa il suo lavoro, l'aggregatore coordina. Il `main()` lavora quasi esclusivamente con l'aggregatore, delegando le operazioni sulle singole entità. Responsabilità tipiche: aggiungere, rimuovere, cercare, ordinare, importare da file.

6. Eccezioni

D: Cos'è un'eccezione in Java? Come si gestisce?

R: Un'eccezione è un **evento anomalo** che interrompe il normale flusso del programma. Si gestisce con il costrutto `try-catch-finally`:

```
try {      // codice che potrebbe generare un'eccezione } catch(TipoEccezione e) {      // gestione dell'eccezione } finally {      // eseguito sempre, con o senza eccezione }
```

D: Qual è la differenza tra eccezioni checked e unchecked?

R:

- **Checked:** il compilatore obbliga a gestirle o a dichiararle con `throws` (es: `IOException`, `SQLException`). Sono eccezioni prevedibili.
- **Unchecked** (`RuntimeException`): non obbligatorie da gestire. Spesso indicano errori di programmazione (es: `NullPointerException`, `ArrayIndexOutOfBoundsException`).

D: Cosa significa propagare un'eccezione? A cosa serve `throws`?

R: Se un metodo non gestisce un'eccezione, la **propaga** al suo chiamante dichiarando `throws TipoEccezione` nella firma. L'eccezione sale la catena delle chiamate fino a trovare un gestore (`catch`). Se non ne trova nessuno, il programma termina con un errore.

D: A cosa serve il blocco `finally`?

R: Il blocco `finally` viene **sempre eseguito**, indipendentemente dal fatto che si sia verificata un'eccezione o meno, e anche se c'è un `return` nel `try`. È tipicamente usato per rilasciare risorse (chiudere file, connessioni DB, ecc.).

D: Si possono avere più `catch` in un `try`? Qual è l'ordine?

R: Sì. I `catch` vengono valutati **dall'alto verso il basso**. Si deve mettere prima il tipo più specifico (sottoclasse) e poi il tipo più generico. Mettere `Exception` come primo `catch` sarebbe un errore logico perché catturerebbe tutto.

7. Strutture Dati (List, Set, Map)

D: Qual è la differenza tra un array e una `List`?

R:

- **Array:** dimensione **fissa**, definita alla creazione. Accesso per indice. Tipo primitivo supportato.
- **List** (`ArrayList`): dimensione **dinamica**, cresce e decresce automaticamente. Richiede tipi boxati (`Integer`, non `int`). Più metodi disponibili (`add`, `remove`, `contains`, ecc.).

D: Cos'è il `for-each`? Cos'è `Iterable`?

R: Il `for-each` (`for(Tipo el : collezione)`) è un costrutto che permette di scorrere qualsiasi oggetto che implementa l'interfaccia `Iterable`. È più leggibile del `for` classico quando non si ha bisogno dell'indice. Funziona con `array`, `List`, `Set` e altre collezioni.

D: Cosa sono i tipi boxati? Perché servono con le collezioni?

R: I tipi boxati sono le versioni **a oggetti** dei tipi primitivi: `Integer (int)`, `Double (double)`, `Boolean (boolean)`, ecc. Le collezioni Java (`List`, `Set`, `Map`) lavorano solo con oggetti, non con primitivi. Java esegue automaticamente il boxing/unboxing (conversione automatica tra primitivo e boxato).

D: Cos'è un `Set`? Come si differenzia da una `List`?

R: Un `Set` è una collezione che **non ammette duplicati e non mantiene l'ordine di inserimento** (in `HashSet`). È utile quando si vogliono elementi unici (es: elenco delle professioni distinte in un censimento). Non permette accesso per indice.

D: Cos'è una `Map`? Come funziona?

R: Una `Map` è una struttura **chiave-valore** (`key → value`). Ogni chiave è univoca; a ogni chiave corrisponde un solo valore. Operazioni principali:

- `put(key, value)`: inserisce/aggiorna.
- `get(key)`: recupera il valore.
- `containsKey(key)`: controlla se la chiave esiste.
- `keySet()`: restituisce tutte le chiavi.
- `entrySet()`: restituisce le coppie chiave-valore.

Implementazione comune: `HashMap`.

8. Ereditarietà

D: Cos'è l'ereditarietà? Come si dichiara in Java?

R: L'ereditarietà permette a una classe (**sottoclasse/figlia**) di acquisire le proprietà e i metodi di un'altra classe (**superclasse/genitore**), estendendola o specializzandola. Si dichiara con `extends`:

```
class Student extends Person { ... }
```

`Student` eredita tutto ciò che è `public` o `protected` da `Person`.

D: Cos'è l'override di un metodo?

R: L'override è la **ridefinizione** di un metodo della superclasse nella sottoclasse. Il metodo ha la stessa firma (nome, parametri, tipo di ritorno) ma un comportamento diverso adatto alla sottoclasse. Si usa l'annotazione `@Override` per chiarezza e sicurezza. Il metodo della superclasse rimane accessibile tramite `super.metodo()`.

D: Cosa significa "tipo formale" e "tipo concreto"?

R:

- **Tipo formale (statico)**: il tipo dichiarato della variabile (es: `Person p`).
- **Tipo concreto (dinamico)**: il tipo effettivo dell'oggetto a cui la variabile punta a runtime (es: `p = new Student()`).

Il tipo formale determina cosa il compilatore permette di fare; il tipo concreto determina quale metodo viene effettivamente chiamato a runtime (polimorfismo).

D: Cos'è il polimorfismo? Come si manifesta?

R: Il polimorfismo permette di trattare oggetti di tipi diversi in modo uniforme tramite un tipo comune (superclasse o interfaccia). A runtime, viene invocato il metodo del **tipo concreto**, non di quello formale. Esempio:

```
Person[] people = { new Student(), new Teacher() }; for(Person p : people)
{ p.describe(); // chiama il metodo corretto in base al tipo reale }
```

D: Cos'è `instanceof`? E il casting?

R:

- `instanceof`: verifica se un oggetto è un'istanza di una certa classe. Restituisce boolean. Es: `if (p instanceof Student)`.
- **Casting**: conversione esplicita del tipo formale. Si usa dopo aver verificato con `instanceof`:

```
if (p instanceof Student) {      Student s = (Student) p;      s.studentSpecificMethod(); }
```

D: Cos'è la classe `Object`? Perché è importante?

R: `Object` è la classe radice di tutta la gerarchia Java. Ogni classe, anche se non dichiarata esplicitamente, estende `Object`. Fornisce metodi base come `toString()`, `equals()`, `hashCode()`. Gli oggetti sono sempre trattati per **riferimento** (a differenza dei primitivi che sono per **valore**).

9. Programmazione a Oggetti Avanzata

D: Cos'è una classe astratta? Quando si usa?

R: Una classe astratta è una classe che **non può essere istanziata** direttamente. Può contenere sia metodi astratti (senza implementazione, che le sottoclassi devono implementare) sia metodi concreti. Si usa quando si vuole fornire una base comune con comportamenti parzialmente definiti.

```
abstract class Shape {      abstract double area(); // le sottoclassi DEVONO implementarlo
void describe() { System.out.println("Sono una forma"); } // concreto }
```

D: Cos'è un'interfaccia? Qual è la differenza con una classe astratta?

R: Un'interfaccia è un **contratto**: definisce cosa un oggetto **sa fare** (metodi), ma non come. Le classi la implementano con `implements`. Differenze principali:

- Un'interfaccia non ha stato (no variabili di istanza, eccetto costanti).
- Una classe può implementare **più interfacce** (non può estendere più classi).
- Da Java 8+, le interfacce possono avere metodi `default` e `static` con implementazione.

D: Cosa sono le interfacce funzionali? Cos'è una lambda?

R:

- **Interfaccia funzionale**: un'interfaccia con un **solo metodo astratto** (es: `Comparator`, `Runnable`). Annotata con `@FunctionalInterface`.
- **Lambda**: un modo compatto per implementare un'interfaccia funzionale. Sintassi: `(parametri) -> corpo`.

Esempio:

```
List<String> nomi = Arrays.asList("Carlo", "Alice", "Bruno"); nomi.sort((a, b) ->
a.compareTo(b));
```

D: Cos'è un tipo generico (Generics)? Perché si usa?

R: I generics permettono di scrivere classi e metodi che lavorano con **tipi parametrici**, specificati dall'utilizzatore. Offrono sicurezza di tipo a compile-time ed eliminano i cast manuali.

```
class Deck<X> {      private List<X> cards = new ArrayList<>();      public void add(X card)
{ cards.add(card); }      public X draw() { return cards.remove(0); } }
```

`Deck<String>`, `Deck<Integer>`, `Deck<Card>` sono tutti tipi validi.

10. Database e SQL

D: Cos'è un database relazionale? Quali sono i suoi elementi fondamentali?

R: Un database relazionale organizza i dati in **tabelle** (relazioni), composte da **righe** (record) e **colonne** (attributi/campi). Ogni tabella ha una **chiave primaria** (PRIMARY KEY) che identifica univocamente ogni riga. Le tabelle sono messe in relazione tramite **chiavi esterne** (FOREIGN KEY).

D: Quali sono i comandi SQL fondamentali per la manipolazione dei dati (DML)?

R:

- `INSERT INTO tabella (col1, col2) VALUES (v1, v2);` → inserimento
- `UPDATE tabella SET col1 = v1 WHERE condizione;` → modifica
- `DELETE FROM tabella WHERE condizione;` → cancellazione
- `SELECT col1, col2 FROM tabella WHERE condizione ORDER BY col1;` → interrogazione

D: Cos'è il `JOIN`? Quali tipi esistono?

R: Il `JOIN` permette di combinare righe di due tabelle basandosi su una condizione (tipicamente la chiave esterna). Tipi principali:

- **INNER JOIN**: restituisce solo le righe con corrispondenza in entrambe le tabelle.
- **LEFT JOIN**: tutte le righe della tabella sinistra, più le corrispondenze a destra (NULL se non c'è corrispondenza).
- **RIGHT JOIN**: speculare al LEFT JOIN.

D: Cos'è il `GROUP BY`? A cosa servono le funzioni aggregate?

R: `GROUP BY` raggruppa le righe con lo stesso valore in una colonna. Le **funzioni aggregate** operano su ogni gruppo:

- `COUNT(*)`: conta le righe
- `SUM(col)`: somma
- `AVG(col)`: media
- `MAX(col)`, `MIN(col)`: massimo e minimo

Esempio: `SELECT professione, COUNT(*) FROM persone GROUP BY professione;`

D: Cos'è l'integrità referenziale? Cosa sono gli orfani?

R: L'integrità referenziale garantisce che una **chiave esterna** punti sempre a una riga esistente nella tabella padre. Un **orfano** è una riga la cui chiave esterna punta a un record padre che non esiste più (es: un ordine il cui cliente è stato cancellato). Le FOREIGN KEY con `ON DELETE CASCADE` eliminano automaticamente gli orfani.

D: Cos'è una VIEW in MySQL?

R: Una VIEW è una **query salvata** che si comporta come una tabella virtuale. Non contiene dati fisici, ma ogni volta che viene interrogata riesegue la query sottostante. È utile per semplificare query complesse e per la sicurezza (nascondere colonne sensibili).

D: Cosa sono le Stored Procedure, le Stored Function e i Trigger?

R:

- **Stored Procedure**: blocco di codice SQL salvato nel DB, richiamabile con `CALL`. Può avere parametri IN/OUT.
 - **Stored Function**: come la Stored Procedure ma restituisce un valore e può essere usata in una `SELECT`.
 - **Trigger**: codice SQL eseguito automaticamente **prima o dopo** un evento (`INSERT`, `UPDATE`, `DELETE`) su una tabella.
-

11. JDBC

D: Cos'è JDBC? Come funziona il flusso di connessione?

R: JDBC (Java Database Connectivity) è l'API Java per comunicare con database relazionali. Il flusso tipico è:

4. **Connection**: aprire la connessione al DB.
5. **Statement** (o **PreparedStatement**): preparare la query.
6. **ResultSet**: eseguire la query e scorrere i risultati.
7. Chiudere le risorse nell'ordine inverso.

```
Connection conn = DriverManager.getConnection(url, user, password); Statement stmt =  
conn.createStatement(); ResultSet rs = stmt.executeQuery("SELECT * FROM persone");  
while(rs.next()) {      System.out.println(rs.getString("nome")); }
```

D: Qual è la differenza tra `Statement` e `PreparedStatement`?

R:

- **Statement**: query statiche, compilate ogni volta. Vulnerabile a **SQL injection**.
- **PreparedStatement**: query parametriche con ? come segnaposto. La query è compilata una volta sola, i parametri sono passati in modo sicuro. **Raccomandato sempre** per input utente.

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM utenti WHERE email = ?");  
ps.setString(1, emailUtente); ResultSet rs = ps.executeQuery();
```

D: Cos'è l'ORM (Object Relational Mapping)? Cos'è il DAO?

R:

- **ORM**: tecnica per **mappare** oggetti Java su righe di tabelle relazionali e viceversa. Elimina il codice ripetitivo di JDBC.
- **DAO (Data Access Object)**: pattern architetturale che **isola** la logica di accesso al database dal resto dell'applicazione. Una classe DAO per ogni entità (es: `PersonDAO`) espone metodi come `findById`, `findAll`, `save`, `delete`.

12. JPA

D: Cos'è JPA? Come si differenzia da JDBC?

R: JPA (Java Persistence API) è uno **standard** Java per l'ORM. Astrae completamente il codice SQL: il programmatore lavora con oggetti Java annotati, e JPA si occupa di generare e gestire le query SQL.

- JDBC → scrivere SQL a mano.
- JPA → lavorare con oggetti, JPA genera l'SQL.

D: Cosa sono le annotazioni principali di JPA?

R:

@Entity: marca la classe come entità persistente mappata su una tabella.

- @Table(name="..."): specifica il nome della tabella.
- @Id: indica la chiave primaria.
- @GeneratedValue: gestione automatica della chiave primaria.
- @Column(name="..."): mappa un campo su una colonna.
- @OneToMany, @ManyToOne, @OneToOne, @ManyToMany: mappano le relazioni.

D: Cos'è l'EntityManager? Cosa sono le entità gestite e non gestite?

R: L'EntityManager è l'interfaccia principale di JPA per le operazioni CRUD:

- `persist(obj)`: salva un nuovo oggetto (INSERT).
- `find(Classe.class, id)`: cerca per chiave primaria (SELECT).
- `merge(obj)`: aggiorna un oggetto (UPDATE).
- `remove(obj)`: elimina (DELETE).

Un'entità è **gestita** (managed) quando è stata recuperata o salvata tramite l'EntityManager nella stessa transazione. È **non gestita** (detached) quando la transazione è chiusa o l'oggetto è stato creato con `new` senza `persist`.

D: Cos'è JPQL?

R: JPQL (Java Persistence Query Language) è un linguaggio di query simile a SQL ma orientato agli **oggetti Java**, non alle tabelle. Opera su nomi di classi e campi, non su nomi di tabelle e colonne.

```
TypedQuery<Person> q = em.createQuery("SELECT p FROM Person p WHERE p.age > :eta",  
Person.class); q.setParameter("eta", 30); List<Person> result = q.getResultList();
```

13. Design Pattern

D: Cos'è un Design Pattern?

R: Un Design Pattern è una **soluzione riutilizzabile** a un problema ricorrente nel design del software. Non è codice pronto, ma un schema concettuale da adattare al contesto specifico. I pattern si dividono in creazionali, strutturali e comportamentali.

D: Cos'è il pattern Factory? Quando si usa?

R: Factory è un pattern **creazionale** che delega la creazione di oggetti a un metodo dedicato (factory method), nascondendo la logica di istanziazione al client. Si usa quando il tipo concreto da creare dipende da condizioni runtime o quando si vuole centralizzare la creazione.

```
Shape s = ShapeFactory.create("circle"); // restituisce un Circle
```

D: Cos'è il pattern Singleton? Come si implementa?

R: Singleton garantisce che di una classe esista **una sola istanza** e ne fornisce un punto di accesso globale.

Implementazione classica:

```
public class Config {    private static Config instance;    private Config() {} //  
costruttore privato    public static Config getInstance() {        if(instance == null)  
instance = new Config();        return instance;    } }
```

D: Cos'è il pattern Adapter?

R: Adapter è un pattern **strutturale** che permette a due interfacce incompatibili di collaborare. Funziona come un adattatore: converte l'interfaccia di una classe esistente nell'interfaccia attesa dal client. Esempio: adattare una libreria di terze parti all'interfaccia usata dal proprio sistema.

D: Cos'è il pattern Proxy?

R: Proxy è un pattern **strutturale** che fornisce un **surrogato** di un altro oggetto per controllarne l'accesso. Usi comuni: lazy loading (creazione ritardata), controllo degli accessi, logging delle operazioni, caching.

D: Cos'è il pattern Facade?

R: Facade è un pattern **strutturale** che fornisce una **interfaccia semplificata** a un sottosistema complesso. Nasconde la complessità interna e offre un'unica classe di accesso per le operazioni più comuni. Riduce le dipendenze tra client e sottosistema.

D: Cos'è il pattern MVC (Model-View-Controller)?

R: MVC è un pattern **architetturale** che separa l'applicazione in tre componenti:

- **Model:** i dati e la logica di business (classi Java, DB).
- **View:** la presentazione all'utente (JSP, HTML, template).
- **Controller:** gestisce le richieste, coordina Model e View (Servlet, @Controller in Spring).

Vantaggi: separazione delle responsabilità, manutenibilità, testabilità.

14. Web (HTML, CSS, Bootstrap, JavaScript)

D: Quali sono i tag HTML fondamentali per i form?

R:

- `<form action="url" method="GET/POST">`: contenitore del form.
- `<input type="text/password/submit/hidden" name="..." value="...">`: campo di input.
- `<select name="..."><option value="...">Testo</option></select>`: menù a tendina.
- `<textarea>`: area di testo multiriga.

D: Cos'è il CSS? Come si applicano i selettori?

R: CSS (Cascading Style Sheets) definisce lo stile visivo degli elementi HTML. Selettori principali:

- **Tag:** `p { color: red; }` → tutti i `<p>`.
- **Classe:** `.mia-classe { font-size: 16px; }` → `class="mia-classe"`.
- **ID:** `#mio-id { ... }` → `id="mio-id"`.
- **Discendenza:** `div p { ... }` → `<p>` dentro `<div>`.

D: Cos'è il Grid System di Bootstrap?

R: Bootstrap divide la riga (`<div class="row">`) in **12 colonne**. Ogni elemento figlio può occuparne un numero specificato con classi come `col-md-6` ($6/12 =$ metà larghezza su schermi medium). Il sistema è **responsive**: le colonne si impilano verticalmente su schermi piccoli.

D: Qual è la differenza tra `var`, `let` e `const` in JavaScript?

R:

- `var`: scope di funzione, soggetto a hoisting. Da evitare nel codice moderno.
- `let`: scope di blocco. Può essere riassegnato.
- `const`: scope di blocco. Non può essere riassegnato (ma l'oggetto puntato può essere mutato).

D: Cos'è il DOM? Come si manipola con JavaScript?

R: Il DOM (Document Object Model) è la rappresentazione ad albero della struttura HTML della pagina. JavaScript lo manipola tramite:

- `document.getElementById("id")`: seleziona un elemento per ID.
 - `document.querySelector(".classe")`: seleziona tramite selettore CSS.
 - `elemento.innerHTML = "..."`: modifica il contenuto.
 - `elemento.style.color = "red"`: modifica lo stile.
 - `elemento.addEventListener("click", funzione)`: gestione eventi.
-

D: Cosa fa jQuery? Cos'è `$(document).ready()`?

R: jQuery è una libreria JavaScript che semplifica la manipolazione del DOM, la gestione degli eventi e le chiamate AJAX. `$(document).ready(function() { ... })` esegue il codice solo quando il DOM è completamente caricato, garantendo che tutti gli elementi esistano prima di manipolarli.

15. Servlet e Web App

D: Cos'è HTTP? Quali sono i verbi principali?

R: HTTP (HyperText Transfer Protocol) è il protocollo di comunicazione del Web. I verbi principali:

- **GET**: recupera una risorsa. Parametri in URL (visibili). Non per dati sensibili.
- **POST**: invia dati al server (es: form). Parametri nel body, non in URL.
- **PUT**: aggiorna una risorsa (usato nei REST API).
- **DELETE**: elimina una risorsa (usato nei REST API).

D: Cos'è una Servlet? Come funziona?

R: Una Servlet è una classe Java che gestisce richieste HTTP. Estende `HttpServlet` e override dei metodi:

- `doGet()`: gestisce le richieste GET.
- `doPost()`: gestisce le richieste POST.

Riceve un `HttpServletRequest` (dati della richiesta) e un `HttpServletResponse` (risposta da costruire). Gira su un **Application Server** (es: Tomcat).

D: Cos'è una JSP (JavaServer Pages)?

R: Una JSP è una pagina HTML con codice Java incorporato. Viene compilata in una Servlet dal server. In un'architettura MVC, la JSP è la **View**: riceve i dati dal Controller (Servlet/Spring) tramite attributi di request/session e li presenta all'utente. `JSTL` (JSP Standard Tag Library) permette di usare tag come `<c:forEach>`, `<c:if>` al posto di codice Java nelle JSP.

D: Cos'è la sessione HTTP? Perché è necessaria?

R: HTTP è un protocollo **stateless**: ogni richiesta è indipendente. La sessione (`HttpSession`) permette di mantenere lo stato tra più richieste dello stesso utente (es: carrello acquisti, utente loggato). I dati sono salvati lato server; il client riceve solo un cookie con l'ID sessione.

16. Spring MVC e REST

D: Cos'è Spring? Cosa offre rispetto alle Servlet?

R: Spring è un framework che semplifica notevolmente lo sviluppo Java EE. Spring MVC offre:

- Gestione automatica delle route tramite annotazioni (@GetMapping, @PostMapping).
- Dependency Injection (DI) integrata.
- Integrazione semplificata con DB, sicurezza, ecc.
- Elimina il boilerplate delle Servlet.

D: Cos'è la Dependency Injection (DI)?

R: La DI è un principio per cui un oggetto non crea le proprie dipendenze, ma le **riceve dall'esterno** (dal framework). In Spring si usa @Autowired o l'iniezione da costruttore. Vantaggi: basso accoppiamento, testabilità, flessibilità.

```
@Controller public class PersonController {    @Autowired    private PersonService service; // Spring inietta l'implementazione }
```

D: Cos'è un Web Service REST? Quali sono i suoi principi?

R: REST (Representational State Transfer) è uno stile architetturale per i Web Service. Principi:

- Basato su **risorse** identificate da URI (es: /api/persone/1).
- Uso dei **verbi HTTP** per le operazioni: GET (leggi), POST (crea), PUT (aggiorna), DELETE (elimina).
- **Stateless**: ogni richiesta contiene tutte le informazioni necessarie.
- Risposta tipicamente in **JSON** o XML.

D: Come si crea un REST endpoint in Spring?

R: Usando @RestController (combina @Controller e @ResponseBody):

```
@RestController @RequestMapping("/api/persone") public class PersonaRestController {    @GetMapping("/{id}")    public Person getById(@PathVariable int id) { ... }    @PostMapping    public Person create(@RequestBody Person p) { ... } }
```

17. AJAX

D: Cos'è AJAX? Qual è il suo vantaggio principale?

R: AJAX (Asynchronous JavaScript And XML) è una tecnica che permette di inviare richieste HTTP al server **in background**, senza ricaricare l'intera pagina. Il vantaggio principale è un'esperienza utente più fluida: solo la parte della pagina che cambia viene aggiornata.

D: Come si fa una chiamata AJAX con jQuery?

R:

```
$.ajax({ url: "/api/dati", method: "GET", success: function(risposta) {  
  // aggiorna il DOM con i dati ricevuti    $("#risultato").html(risposta);    },  
error: function(err) { console.log("Errore: " + err);    } });
```

Con fetch (API moderna nativa):

```
fetch("/api/dati")    .then(res => res.json())    .then(data => console.log(data));
```